Politecnico di Torino

Lab report for the course Qubit Electronics

Master degree in Quantum Engineering

Authors: Group 3

Riola Leonardo Antonio, Donati Davide, Girotto Giacomo

April 27, 2026

Contents

1 Mesh generation **1**
1.1 Defining dimensions 1
1.2 Mesh structures and solids 1
1.3 File formats and export 2

2 Analysis of the QTCAD Script **2**
2.1 Mesh import and unit conversion 2
2.2 Definition of the device and operating temperature 3
2.3 Region names, assigned materials, and correspondence with the mesh 3
2.4 Boundary conditions and definition of the dot region 4
2.5 Poisson and Schrodinger Solver Setup and Execution 5
2.6 Physical interpretation 5

1. Mesh generation

1.1 Defining dimensions

In order to initialize the structure of the Double Quantum Dot, in the `dqdfdsoi.geo` file are defined the main dimensions that caracterize the device:
First, the dimensions of the domain along the x direction are defined through the variable `domain_width` and `channel_width`, where the latter is the width of the portion of the device where the quantum dots will be defined. The dimensions are set equal to 60nm and 40nm, respectively.

Then, along the y direction, the lenghts of the plunger gates, barrier gates, source/drain and the gaps between them are defined, which are summerized in the following scheme (1.1):

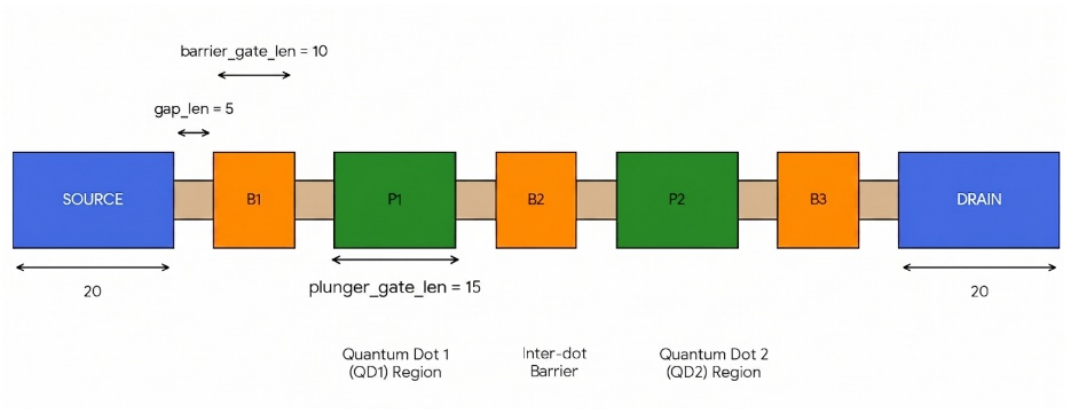

Figure 1.1: Scheme of the double quantum dot structure along the y direction, all the dimensions are in nm.

Finally, the thicknesses along the z direction of the different layers. Starting from the bottom of the device, we have in order:

- `box_thick`: set equal to 10nm, it is the thickness of the oxide below the silicon structure;
- `film_thick`: set equal to 10nm, represents the thickness of the silicon layer where the quantum dots will be formed;
- `gate_oxide_thick`: set equal to 2nm, it is the thickness of the oxide layer below the gates.

1.2 Mesh structures and solids

The geometry is constructed by defining several rectangles that represent the functional parts of the Double Quantum Dot. The command used is `Rectangle(tag) = {x, y, z, dx, dy}`, where the coordinates define the lower-left corner and `dx, dy` the dimensions. First, a rectangle is defined to represent the main domain of the simulation and above it the rectangles representing source, drain and the channel are costructed. Then, a sequence of rectangles is generated through a `For` loop to define the barrier and plunger gates. These rectangles initially overlap with the main channel and the simulation domain. To ensure a conformal mesh (where nodes match perfectly at the interfaces), the `BooleanFragments` command is applied. This operator "slices" all the surfaces at their intersections, creating a mosaic of non-overlapping pieces.

Solids are defined through **Extrusion**. This process takes a 2D surface and "pushes" it along the z-axis, automatically creating the volume and the lateral boundary surfaces. In our simulation, this method is used to create the oxide layer below the gates, through the line:
`Extrude {0, 0, gate_oxide_thick} {Surface{2:36}; Layers {2};}`, where the surfaces from 2 to 36 represents the surfaces generated after the `BooleanFragments` command and `Layers{2}`explicitly divides the thickness of the extruded volume into two layers of prismatic elements for the subsequent mesh generation. This ensures that the mesh nodes are consistently aligned along the z-direction, providing the minimum vertical resolution required for the numerical convergence of the Poisson and Schrödinger solvers. However, one could define a solid manually without extrusion by following these steps:

1. Define the **Surfaces** that form the boundary of the object;
2. Group these surfaces into a **Surface Loop**, which represents a closed shell;
3. Define a **Volume** by referencing the ID of the Surface Loop.

In our case, the extrusion is the most efficient method because it guarantees that the volumes of the different parts of the device are perfectly stacked and aligned.

1.3 File formats and export

At the end of the script, the structure is saved in two different formats, each serving a specific purpose in the QTCAD workflow:

- **.msh2 (Gmsh Mesh Version 2.0):** This is the standard file format for storing the generated mesh (nodes, elements, and physical groups). It is the file that the QTCAD solver will actually read to perform the finite element calculations. The version 2.0 is often preferred for compatibility with various scientific libraries.
- **.geo_unrolled:** This is a "flat" version of the geometry script. Since the original code uses loops (`For`) and variables, the unrolled version expands everything into raw, explicit Gmsh commands. It is extremely useful for debugging, as it allows the user to see exactly which tags and IDs were assigned to every single point, line, or surface created during the generation process. However, The use of the `OpenCASCADE` kernel and advanced boolean operations like `BooleanFragments` leads to a `.geo_unrolled` file that contains a `Merge` command pointing to a `.xao` file, where Gmsh serializes the internal CAD state into the `.xao` (OpenCASCADE ASCII) format, which stores the Boundary Representation (B-Rep) of the fragmented structure.

2. Analysis of the QTCAD Script

2.1 Mesh import and unit conversion

The first relevant part of `double_dot_stability.py` is the import of the mesh generated with Gmsh:

Listing 2.1: Mesh loading in the QTCAD script.

```
scaling = 1e-9
mesh = Mesh(scaling, str(path_mesh / "dqdfdsoi.msh2"))
```

These lines appear at lines 45–46 of the script. Their role is to load the three-dimensional structure from the file `dqdfdsoi.msh2` and convert its coordinates into SI units. This conversion is necessary because the geometry file `dqdfdsoi.geo` is written in nanometers, as explicitly stated in line 3 of the mesh source. Therefore, the factor `1e-9` converts all mesh coordinates from nanometers to meters before they are used by QTCAD.

From the point of view of the `Mesh` class, the two arguments have different roles:

- the first argument, `scaling`, is the geometric conversion factor applied to the coordinates of the mesh nodes;
- the second argument is the path of the Gmsh file to be imported, here given as the string corresponding to `meshes/dqdfdsoi.msh2`.

This means that the same mesh can be reused in QTCAD without rewriting the geometry, provided that the correct spatial scaling is supplied at import time.

2.2 Definition of the device and operating temperature

Once the mesh has been loaded, the script creates the device object:

Listing 2.2: Device creation and temperature assignment.

```
dvc = Device(mesh, conf_carriers='e')
dvc.set_temperature(0.1)
```

These lines correspond to lines 49–50. The first line constructs a `Device` object on top of the imported mesh. The first argument is the mesh itself, which contains the geometry and the physical regions. The keyword argument `conf_carriers='e'` specifies that the confined carriers are electrons. This is consistent with the fact that the script later studies the conduction-band profile and the electronic bound states of the double quantum dot.

The second line sets the operating temperature to 0.1 K. Such a cryogenic temperature is coherent with the physical context of semiconductor quantum dots, which are typically operated at millikelvin temperatures to suppress thermal excitations and observe quantum phenomena.

2.3 Region names, assigned materials, and correspondence with the mesh

After the device object is created, the script assigns materials to the physical regions declared in the mesh. This correspondence is essential: the names used in `double_dot_stability.py` must match the physical volume names defined in `dqdfdsoi.geo`, otherwise QTCAD would not be able to associate the material parameters with the correct parts of the structure.

Region name	**Material**	**Meaning**	**Script line**	**Mesh line**
`oxide`	`SiO2`	Lateral oxide region	53	76
`oxide_dot`	`SiO2`	Oxide around active dot region	54	78
`gate_oxide`	`HfO2`	Gate dielectric	55	63
`gate_oxide_dot`	`HfO2`	Gate dielectric in active region	56	62
`buried_oxide`	`SiO2`	BOX layer	57	99
`buried_oxide_dot`	`SiO2`	BOX below active region	58	100
`channel`	`Si`	Silicon channel	59	75
`channel_dot`	`Si`	Silicon region where the dots form	60	77
`source`	`Si`, n-doped	Source reservoir	61	73
`drain`	`Si`, n-doped	Drain reservoir	62	74

Table 2.1: Correspondence between script regions and physical volumes defined in the mesh.

The table shows that the naming is intentionally identical in the Python script and in the mesh. The doped regions are `source` and `drain`, both declared as silicon with `ndoping=1e20*1e6`. The factor `1e6`

converts the doping from cm^{-3} to m^{-3}, so the implemented concentration is $10^{20} cm^{-3}$.

It is also worth noting that the active dot region is not limited to `channel_dot`. The list later used for the quantum-dot calculation also includes the dielectric regions `oxide_dot`, `gate_oxide_dot`, and `buried_oxide_dot`. This choice preserves the local electrostatic environment of the dots instead of isolating only the silicon volume.

Listing 2.3: Material assignment to the main regions.

```
dvc.new_region("oxide", mt.SiO2)
dvc.new_region("oxide_dot", mt.SiO2)
dvc.new_region("gate_oxide", mt.HfO2)
dvc.new_region("gate_oxide_dot", mt.HfO2)
dvc.new_region("buried_oxide", mt.SiO2)
dvc.new_region("buried_oxide_dot", mt.SiO2)
dvc.new_region("channel", mt.Si)
dvc.new_region("channel_dot", mt.Si)
dvc.new_region("source", mt.Si, ndoping=1e20*1e6)
dvc.new_region("drain", mt.Si, ndoping=1e20*1e6)
```

2.4 Boundary conditions and definition of the dot region

The next stage of the script defines the electrostatic boundary conditions. The gate boundaries are introduced through the method `new_gate_bnd`, for example:

Listing 2.4: Gate boundaries in the QTCAD script.

```
Ew = mt.Si.Eg/2 + mt.Si.chi
dvc.new_gate_bnd("barrier_gate_1_bnd", barrier_gate_1_bias, Ew)
```

The arguments of `new_gate_bnd` are:

- the name of the physical surface on which the boundary condition is applied;
- the applied gate voltage;
- the metal work function, here chosen as `Ew`, namely the silicon midgap reference $E_g/2+\chi$.

In this script, the boundary names are again consistent with the mesh file, where the same physical surfaces are declared. The gate, contact, and back-gate boundaries are summarized below.

Boundary name	**Type**	**Bias [V]**	**Script line**	**Mesh line**
`barrier_gate_1_bnd`	gate	0.5	74	55
`plunger_gate_1_bnd`	gate	0.6	75	56
`barrier_gate_2_bnd`	gate	0.5001	76	57
`plunger_gate_2_bnd`	gate	0.6	77	58
`barrier_gate_3_bnd`	gate	0.5	78	59
`source_bnd`	ohmic	–	79	69
`drain_bnd`	ohmic	–	80	70
`back_gate_bnd`	frozen	-0.5	81–82	93–96

Table 2.2: Boundary conditions used in the QTCAD script and their correspondence with the mesh.

The dot region is then explicitly identified at lines 85–86:

Listing 2.5: Definition of the active dot region.

```
dot_region_list = ["oxide_dot", "gate_oxide_dot",
                   "buried_oxide_dot", "channel_dot"]
dvc.set_dot_region(dot_region_list)
```

Therefore, the materials that compound the dot region are:

- `Si` for `channel_dot`;
- `SiO2` for `oxide_dot`;
- `HfO2` for `gate_oxide_dot`;
- `SiO2` for `buried_oxide_dot`.

This confirms that the active region used by the solvers is not only the silicon island itself, but the local portion of the structure that determines the confinement and dielectric screening around it.

2.5 Poisson and Schrodinger Solver Setup and Execution

The concluding part of the script is devoted to the definition of the Poisson and Schrodinger solvers, the assignment of their parameters, and the execution of the corresponding calculations. The structure of the script is summarized in Table 2.3.

Operation	**Line(s)**
Instantiation of Poisson solver parameters: `params_poisson = PoissonSolverParams()`	89
Definition of Poisson parameter values	90–100
Definition of the number of single-particle states: `num_states = 4`	103
Instantiation of Schrodinger solver parameters: `params_schrod = SchrodingerSolverParams()`	106
Assignment of the number of states: `params_schrod.num_states = num_states`	107
Instantiation of the Poisson solver: `poisson_slv = PoissonSolver(...)`	110–111
Solution of the Poisson problem: `poisson_slv.solve()`	114
Transfer of the electrostatic potential into the confinement potential: `dvc.set_V_from_phi()`	118
Creation of the submesh for the active region: `submesh = SubMesh(...)`	121
Creation of the subdevice: `subdvc = SubDevice(...)`	124
Instantiation of the Schrodinger solver: `schrod_solver = SchrodingerSolver(...)`	127
Solution of the Schrodinger problem: `schrod_solver.solve()`	130

Table 2.3: Main steps of the Poisson–Schrodinger workflow in `double_dot_stability.py`.

This sequence shows that the script first solves the electrostatic problem on the complete device and only afterwards solves the quantum problem on the restricted active region. The intermediate line `dvc.set_V_from_phi()` is the key step that connects the two parts of the simulation, because it converts the electrostatic solution into the confinement potential used for the electronic eigenstates.

2.6 Physical interpretation

Overall, the script implements a two-step description of the device. The complete geometry is first used to reconstruct the electrostatic environment generated by the gates, the contacts, and the dielectric stack. This

electrostatic solution is then converted into the confinement potential used in the Schrodinger calculation, which is restricted to the active region of the double quantum dot.

The script is therefore organized so that the physical definition of the device directly feeds the calculation of its lowest electronic states.